

subject	title	check ☐ ☐ ☐ ☐ ☐
수요일스터디	수요일스터디 - C++ 조사 주제 4개 개념정리	date

2026.06.12

C++ 조사 주제 4개 쉬운 예제 정리

전체 핵심

이 문서는 다음 4가지 주제를 쉬운 예제와 코드로 정리한 자료입니다.

1. C와 C++ 목차별 차이점 2. C++ Call by Value, Call by Address, Call by Reference 3. 객체지향언어 C++과 Java의 차이점 4. STL이란 무엇인가

핵심은 "언어마다 문제를 해결하는 방식이 다르다"는 것입니다. C는 순서대로 처리하는 절차 중심이고, C++은 C의 기능에 클래스와 객체지향, STL을 더한 언어입니다. Java는 JVM 위에서 실행되어 여러 환경에서 안정적으로 동작하기 쉽게 만든 언어입니다.

1. C와 C++ 목차별 차이점

한 줄 핵심

C는 절차지향 중심 언어이고, C++은 C의 기능을 포함하면서 클래스, 객체, 상속, 함수 오버로딩, 참조, STL 같은 기능을 추가한 언어입니다.

쉬운 비유

C는 "요리 순서표"와 비슷합니다.

1. 재료를 씻는다.
2. 재료를 자른다.
3. 냄비에 넣는다.
4. 끓인다.
5. 접시에 담는다.

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

정해진 순서대로 함수가 실행되며 문제를 해결합니다.

C++은 "요리 도구와 재료를 묶어서 관리하는 주방 시스템"에 가깝습니다. 예를 들어 'Book'이라는 클래스를 만들면 책의 번호, 제목 같은 데이터와 책 정보를 출력하는 기능을 하나로 묶을 수 있습니다.

Book = 책 번호 + 책 제목 + 책 출력 기능
Student = 이름 + 학번 + 점수 계산 기능
Account = 계좌번호 + 잔액 + 입금/출금 기능

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

subject	title	check ☐ ☐ ☐ ☐ ☐
수요일스터디	수요일스터디 - C++ 조사 주제 4개 개념정리	date

목차별 비교

목차	C	C++
프로그래밍 방식	절차지향 중심	절차지향 + 객체지향
데이터 묶기	`struct`	`struct`, `class`
함수 이름	같은 이름의 함수 여러 개 사용 어려움	함수 오버로딩 가능
문자열	`char 배열`	`string` 사용 가능
입출력	`printf`, `scanf`	`cout`, `cin`
메모리 관리	`malloc`, `free`	`new`, `delete`
객체지향	직접 지원 약함	클래스, 상속, 다형성 지원
자료구조	직접 구현하는 경우 많음	STL 사용 가능
사용 예	시스템, 임베디드, C 과제	큰 프로그램, 자료구조, 알고리즘, 객체 설계

C 방식 예시

C에서는 데이터는 `struct`로 묶고, 기능은 함수로 따로 만듭니다.

```
#include <stdio.h>

typedef struct {
    int id;
    char title[100];
} Book;

void printBook(Book b) {
    printf("%d %s\n", b.id, b.title);
}
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

여기서 `Book`은 책 정보를 담는 상자이고, `printBook()`은 그 상자를 받아 출력하는 함수입니다.

subject	title	check ☐ ☐ ☐ ☐ ☐
수요일스터디	수요일스터디 - C++ 조사 주제 4개 개념정리	date

C++ 방식 예시

C++에서는 데이터와 기능을 클래스 안에 함께 넣을 수 있습니다.

```
#include <iostream>
#include <string>
using namespace std;

class Book {
private:
    int id;
    string title;

public:
    Book(int id, string title) : id(id), title(title) {}

    void printBook() {
        cout << id << " " << title << endl;
    }
};
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

‘Book’ 객체가 자기 데이터와 자기 기능을 함께 가집니다.

struct와 class 차이

C의 ‘struct’는 여러 데이터를 하나로 묶는 상자에 가깝습니다.

C++의 ‘class’는 데이터와 기능을 함께 묶는 설계도입니다.

쉬운 예:

```
C struct Book
= 책 번호와 제목을 담는 종이 양식

C++ class Book
= 책 번호, 제목, 출력 기능까지 포함한 책 관리 도구
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

subject	title	check ☐ ☐ ☐ ☐ ☐
수요일스터디	수요일스터디 - C++ 조사 주제 4개 개념정리	date

문자열 차이

C 문자열:

```
char name[20] = "Kim";
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

C++ 문자열:

```
string name = "Kim";
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

C에서는 문자열이 문자 배열이므로 `strcmp`, `strcpy`, `strlen` 같은 함수를 자주 사용합니다. C++에서는 `string` 클래스를 사용해 문자열을 더 편하게 다룰 수 있습니다.

입출력 차이

C:

```
printf("Age: %d\n", age);  
scanf("%d", &age);
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

C++:

```
cout << "Age: " << age << endl;  
cin >> age;
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

C는 `%d`, `%s` 같은 형식 지정자를 사용합니다. C++은 `cout`, `cin`을 사용해 흐름처럼 데이터를 입출력합니다.

메모리 관리 차이

C:

```
int* arr = (int*)malloc(sizeof(int) * 5);  
free(arr);
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

C++:

```
int* arr = new int[5];  
delete[] arr;
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

C++에서는 객체를 만들 때 생성자가 실행되고, 객체가 사라질 때 소멸자가 실행될 수 있습니다. 그래서 객체 단위의 관리가 더 자연스럽습니다.

subject	title	check ☐ ☐ ☐ ☐ ☐
수요일스터디	수요일스터디 - C++ 조사 주제 4개 개념정리	date

최종 암기 문장

C는 함수와 순서 중심으로 문제를 해결하는 언어입니다. C++은 C를 바탕으로 클래스, 객체, 상속, 다형성, STL 같은 기능을 더한 언어입니다. C가 요리 순서표라면, C++은 재료와 도구와 기능을 묶어 관리하는 주방 시스템에 가깝습니다.

2. C++ Call by Value, Address, Reference

한 줄 핵심

함수에 값을 넘길 때 복사본을 줄지, 주소를 줄지, 별명을 줄지 정하는 개념입니다.

쉬운 비유

친구에게 내 노트를 수정해달라고 부탁한다고 생각해 봅시다.

Call by Value

= 노트 복사본을 준다.

친구가 복사본에 낙서해도 내 원본 노트는 그대로다.

Call by Address

= 내 노트가 있는 책상 위치를 알려준다.

친구가 그 위치로 가서 원본 노트를 직접 고친다.

Call by Reference

= 내 노트에 다른 이름을 붙여준다.

친구가 그 이름으로 고치면 원본 노트가 고쳐진다.

Const Reference

= 원본 노트를 보여주지만 수정은 못 하게 한다.

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

subject	title	check ☐ ☐ ☐ ☐ ☐
수요일스터디	수요일스터디 - C++ 조사 주제 4개 개념정리	date

Call by Value

값을 복사해서 함수에 전달합니다.

```
#include <iostream>
using namespace std;

void change(int x) {
    x = 10;
}

int main() {
    int num = 5;
    change(num);
    cout << num << endl;
}
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

결과 결과

5

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

`num`의 복사본이 `x`로 전달됩니다. 함수 안에서 `x`를 10으로 바꿔도 원본 `num`은 그대로 5입니다.

Call by Value를 쓰는 경우

원본 값을 보호하고 싶을 때 사용합니다.

```
int square(int x) {
    return x * x;
}
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

이 함수는 입력값을 계산에만 사용합니다. 원본을 바꿀 필요가 없으므로 Call by Value가 적절합니다.

subject	title	check ☐ ☐ ☐ ☐ ☐
수요일스터디	수요일스터디 - C++ 조사 주제 4개 개념정리	date

Call by Address

변수의 주소를 함수에 전달합니다. 포인터를 사용합니다.

```
#include <iostream>
using namespace std;

void change(int* x) {
    *x = 10;
}

int main() {
    int num = 5;
    change(&num);
    cout << num << endl;
}
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

결과 결과

10

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

`&num`은 `num`의 주소입니다. 함수 안에서 `\*x = 10`을 하면 그 주소에 있는 원본 값이 바뀝니다.

Call by Address를 쓰는 경우

원본 값을 직접 바꿔야 할 때 사용합니다.

```
void swapValue(int* a, int* b) {
    int temp = *a;
    *a = *b;
    *b = temp;
}
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

호출:

```
swapValue(&x, &y);
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

주소를 넘겼기 때문에 함수 안에서 `x`와 `y`의 원본 값을 바꿀 수 있습니다.

subject	title	check ☐ ☐ ☐ ☐ ☐
수요일스터디	수요일스터디 - C++ 조사 주제 4개 개념정리	date

Call by Reference

참조자를 사용해서 원본 변수에 다른 이름을 붙입니다.

```
#include <iostream>
using namespace std;

void change(int& x) {
    x = 10;
}

int main() {
    int num = 5;
    change(num);
    cout << num << endl;
}
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

결과 결과

10

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

`x`는 `num`의 별명입니다. 함수 안에서 `x`를 바꾸면 원본 `num`도 바뀝니다.

Call by Reference를 쓰는 경우

C++에서 원본 값을 바꾸고 싶을 때 포인터보다 간단하게 사용할 수 있습니다.

```
void swapValue(int& a, int& b) {
    int temp = a;
    a = b;
    b = temp;
}
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

호출:

```
swapValue(x, y);
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

포인터처럼 `&x`, `\*a`를 많이 쓰지 않아도 되어 코드가 읽기 쉽습니다.

subject	title	check ☐ ☐ ☐ ☐ ☐
수요일스터디	수요일스터디 - C++ 조사 주제 4개 개념정리	date

Const Reference

복사는 피하고 싶지만 원본은 바꾸면 안 될 때 사용합니다.

```
void printName(const string& name) {  
    cout << name << endl;  
}
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

`string` 같은 큰 객체를 값으로 넘기면 복사 비용이 생길 수 있습니다. `const string&`로 넘기면 복사를 줄이고, 함수 안에서 원본을 수정하지 못하게 막을 수 있습니다.

비교 표

방식	쉬운 설명	원본 변경	특징
Call by Value	복사본 전달	불가능	안전하지만 복사 비용 가능
Call by Address	주소 전달	가능	포인터 사용, C에서도 가능
Call by Reference	별명 전달	가능	C++ 방식, 문법이 간단
Const Reference	수정 불가 별명 전달	불가능	큰 객체를 안전하고 빠르게 전달

최종 암기 문장

Value는 복사본, Address는 주소, Reference는 별명입니다. Value는 원본 보호, Address와 Reference는 원본 변경, Const Reference는 복사 비용을 줄이면서 원본을 보호하기 위해 사용합니다.

3. 객체지향언어 C++과 Java의 차이점

한 줄 핵심

C++과 Java는 둘 다 객체지향을 지원하지만, C++은 성능과 직접 제어가 강하고 Java는 JVM 기반의 안정성, 이식성, 유지보수가 강합니다.

기본 방향 차이

C++은 개발자가 프로그램의 동작과 메모리를 세밀하게 제어할 수 있도록 만든 언어입니다. 그래서 실행 속도와 자원 관리가 중요한 프로그램에 강하지만, 그만큼 개발자가 신경 써야 할 부분도 많습니다.

Java는 여러 운영체제에서 같은 프로그램을 안정적으로 실행하고, 큰 규모의 프로그램을 유지보수하기 쉽게 만드는 데 강한 언어입니다. 직접 제어할 수 있는 부분은 C++보다 줄어들지만, JVM과 GC가 실행 환경과 메모리 관리를 많이 도와줍니다.

C++ = 직접 제어, 빠른 성능, 높은 자유도

Java = JVM 기반 실행, 자동 메모리 관리, 높은 유지보수성

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

subject	title	check ☐ ☐ ☐ ☐ ☐
수요일스터디	수요일스터디 - C++ 조사 주제 4개 개념정리	date

실행 방식 차이

C++ 실행 흐름:

```
C++ 코드
-> 컴파일
-> 운영체제와 CPU에 맞는 실행 파일
-> 직접 실행
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

Java 실행 흐름:

```
Java 코드
-> 컴파일
-> bytecode(.class)
-> JVM에서 실행
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

C++은 운영체제에 맞는 실행 파일로 만들어집니다. Windows용으로 만든 실행 파일을 Linux에서 그대로 실행하기는 어렵습니다.

Java는 bytecode로 컴파일되고 JVM이 실행합니다. 운영체제 별 JVM만 있으면 같은 Java 프로그램을 여러 환경에서 실행하기 쉽습니다.

정리하면 C++은 컴파일 결과가 운영체제와 CPU에 더 직접적으로 연결되고, Java는 JVM이라는 중간 실행 환경을 한 번 거칩니다. 이 차이 때문에 C++은 실행 성능과 하드웨어 제어에 유리하고, Java는 이식성과 배포 편의성에 유리합니다.

JVM이란?

JVM은 Java Virtual Machine의 약자입니다. Java 프로그램을 실행해주는 가상의 컴퓨터라고 생각하면 됩니다.

JVM의 역할은 Java bytecode를 실제 운영체제와 CPU에서 실행될 수 있도록 처리하는 것입니다.

```
Java 프로그램은 bytecode라는 공통 언어로 말한다.
JVM은 그 말을 Windows, Linux, macOS 환경에 맞게 실행한다.
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

그래서 Java는 "Write Once, Run Anywhere"라는 표현으로 자주 설명됩니다. 한 번 작성한 Java 코드를 각 운영체제용 JVM 위에서 실행할 수 있다는 뜻입니다.

subject	title	check ☐ ☐ ☐ ☐ ☐
수요일스터디	수요일스터디 - C++ 조사 주제 4개 개념정리	date

메모리 관리 차이

C++은 개발자가 직접 메모리를 관리할 수 있습니다. 필요한 순간에 `new`로 메모리를 할당하고, 더 이상 필요 없으면 `delete`로 해제할 수 있습니다.

```
int* p = new int;
*p = 10;
delete p;
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

Java는 객체를 만들 수 있지만 `delete`로 직접 지우지 않습니다.

```
Person p = new Person();
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

Java에서는 Garbage Collector, 즉 GC가 사용하지 않는 객체를 자동으로 정리합니다.

C++은 직접 제어가 가능하기 때문에 메모리를 매우 효율적으로 사용할 수 있지만, 해제를 잊으면 메모리 누수가 발생할 수 있습니다. Java는 GC가 자동으로 정리해주기 때문에 안전하고 편하지만, GC가 동작하는 과정에서 순간적인 성능 비용이 생길 수 있습니다.

메모리 관리 비교

구분	C++	Java
메모리 할당	`new`	`new`
메모리 해제	`delete` 직접 사용 가능	GC가 자동 정리
장점	세밀한 제어, 성능 최적화	안전성, 편의성
단점	메모리 누수 위험	GC 오버헤드 가능

포인터와 참조 차이

C++은 포인터를 직접 사용할 수 있습니다. 변수의 주소를 얻고, 그 주소를 통해 값을 읽거나 수정할 수 있습니다.

```
int a = 10;
int* p = &a;
cout << *p << endl;
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

Java도 객체를 참조로 다루지만, C++처럼 주소 계산을 하거나 포인터 연산을 직접 하지는 않습니다.

```
Person p = new Person();
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

Java의 `p`는 객체를 가리키지만, 개발자가 메모리 주소를 직접 계산하거나 이동시키는 방식은 아닙니다.

즉 C++의 포인터는 메모리 주소를 직접 다루는 도구이고, Java의 참조는 객체에 접근하기 위한 안전한 연결 이름에 가깝습니다. Java는 포인터 연산을 막아서 메모리 오류 가능성을 줄입니다.

subject	title	check ☐ ☐ ☐ ☐ ☐
수요일스터디	수요일스터디 - C++ 조사 주제 4개 개념정리	date

객체지향 구조 차이

C++은 일반 함수와 클래스를 모두 사용할 수 있습니다.

```
int add(int a, int b) {  
    return a + b;  
}  
  
class Student {  
public:  
    void print() {  
        cout << "student" << endl;  
    }  
};
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

Java는 대부분의 코드가 클래스 안에 들어갑니다.

```
public class Main {  
    public static void main(String[] args) {  
        System.out.println("Hello");  
    }  
}
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

C++은 자유도가 높고, Java는 클래스 중심 구조가 더 강합니다.

그래서 C++은 C처럼 함수 중심으로도 작성할 수 있고, 필요하면 클래스 중심으로 설계할 수도 있습니다. Java는 프로그램의 기본 단위가 클래스이므로 객체지향 구조를 더 강하게 따르는 편입니다.

subject	title	check ☐ ☐ ☐ ☐ ☐
수요일스터디	수요일스터디 - C++ 조사 주제 4개 개념정리	date

상속 차이

C++은 클래스 다중 상속을 지원합니다.

```
class Printer {
public:
    void print() {}
};

class Scanner {
public:
    void scan() {}
};

class MultiFunctionMachine : public Printer, public Scanner {
};
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

Java는 클래스 다중 상속을 허용하지 않습니다.

```
class Student extends Person {
}
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

대신 여러 인터페이스를 구현할 수 있습니다.

```
class MultiFunctionMachine implements Printable, Scannable {
}
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

즉 C++은 여러 부모 클래스의 구현을 직접 물려받을 수 있습니다. Java는 부모 클래스는 하나만 상속받고, 여러 역할은 인터페이스로 약속합니다. Java가 이런 방식을 쓰는 이유는 다중 상속에서 발생할 수 있는 충돌 문제를 줄이기 위해서입니다.

subject	title	check ☐ ☐ ☐ ☐ ☐
수요일스터디	수요일스터디 - C++ 조사 주제 4개 개념정리	date

오버라이딩 차이

C++에서는 부모 포인터로 자식 객체를 다룰 때 실제 자식 함수가 실행되게 하려면 `virtual`이 중요합니다.

```
class Animal {
public:
    virtual void sound() {
        cout << "sound" << endl;
    }
};
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

Java에서는 일반 인스턴스 메서드가 기본적으로 동적 바인딩됩니다.

```
class Animal {
    void sound() {
        System.out.println("sound");
    }
}
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

그래서 Java는 C++처럼 매번 `virtual`을 붙이지 않아도 오버라이딩된 메서드가 자연스럽게 호출됩니다.

예를 들어 `Animal` 타입 변수에 `Dog` 객체를 넣고 `sound()`를 호출할 때, C++은 부모 함수에 `virtual`이 있어야 `Dog`의 함수가 실행됩니다. Java는 일반 인스턴스 메서드라면 기본적으로 실제 객체 기준으로 메서드를 찾아 실행합니다.

단, Java에서도 `static`, `final`, `private` 메서드는 일반적인 오버라이딩 대상이 아닙니다. 시험이나 면접에서는 "C++은 `virtual`이 핵심이고, Java는 기본적으로 동적 바인딩"이라고 기억하면 좋습니다.

subject	title	check ☐ ☐ ☐ ☐ ☐
수요일스터디	수요일스터디 - C++ 조사 주제 4개 개념정리	date

전체 비교 표

구분	C++	Java
실행 방식	운영체제와 CPU에 맞는 기계어 실행 파일	bytecode를 JVM에서 실행
핵심 장점	성능, 직접 제어	이식성, 안정성, 유지보수
실행 환경	실행 파일이 환경에 직접 의존	JVM이 있으면 여러 환경에서 실행 쉬움
메모리 관리	개발자가 직접 가능	GC 자동 관리
메모리 해제	`delete`로 직접 해제 가능	개발자가 직접 `delete`하지 않음
포인터	주소 접근과 포인터 연산 가능	직접 포인터 연산 없음
객체지향	절차지향 + 객체지향	클래스 중심 객체지향
다중 상속	클래스 다중 상속 가능	클래스 다중 상속 불가, 인터페이스 사용
오버라이딩	부모 포인터/참조에서는 `virtual` 중요	일반 인스턴스 메서드는 기본적으로 동적 바인딩
대표 프레임워크	Qt, Unreal Engine 등	Spring, Android 기존 Java 코드 등
주요 분야	시스템, 임베디드, 성능 중요한 프로그램	웹 서버, 기업 시스템, Android 기존 코드

성능과 사용 분야 차이

C++은 실행 파일이 하드웨어와 운영체제에 더 가깝게 동작하고, 메모리 관리도 직접 할 수 있어 성능을 끝까지 끌어올리기 좋습니다. 대신 실수하면 메모리 누수, 잘못된 포인터 접근, 객체 수명 문제 같은 오류가 발생할 수 있습니다.

Java는 JVM 위에서 실행되고 GC가 메모리를 관리하므로 개발자가 시스템 세부 사항을 직접 다루는 부담이 줄어듭니다. 그래서 대규모 서버, 기업 시스템, Spring 기반 백엔드처럼 오래 운영하고 유지보수해야 하는 프로그램에 많이 사용됩니다.

언제 C++이 유리한가

성능이 매우 중요하거나 하드웨어 가까운 제어가 필요하다면 C++이 유리합니다.

예시:

- 임베디드 장치
- 로봇 제어
- 자동차 제어 장치
- 운영체제에 가까운 시스템 프로그램
- 실시간 처리가 중요한 프로그램

subject	title	check ☐ ☐ ☐ ☐ ☐
수요일스터디	수요일스터디 - C++ 조사 주제 4개 개념정리	date

언제 Java가 유리한가

여러 환경에서 안정적으로 오래 운영해야 하거나, 대규모 서버 개발과 유지보수가 중요하다면 Java가 유리합니다.

예시:

- 웹 서버
- 쇼핑몰 주문 시스템
- 회사 업무 시스템
- 은행 계좌 시스템
- Spring 기반 REST API 서버
- Android 기존 Java 코드 유지보수

최종 암기 문장

C++은 빠른 실행, 메모리 직접 제어, 포인터, 다중 상속, 'virtual' 기반 다형성에 강합니다. Java는 JVM 기반 실행, GC 자동 메모리 관리, 클래스 중심 구조, 인터페이스, 기본 동적 바인딩을 통해 안정적인 유지보수에 강합니다.

4. STL이란 무엇인가

한 줄 핵심

STL은 C++에서 자주 쓰는 자료구조와 알고리즘을 미리 만들어둔 표준 라이브러리입니다.

기본 개념

vector = 크기가 자동으로 늘어나는 배열
list = 노드들이 앞뒤로 연결된 연결 리스트
stack = LIFO 구조, 나중에 들어온 값이 먼저 나감
queue = FIFO 구조, 먼저 들어온 값이 먼저 나감
map = key와 value를 한 쌍으로 저장
set = 중복을 허용하지 않고 값을 저장
sort = 범위 안의 데이터를 정렬하는 알고리즘

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

STL의 3대 구성

구성	의미	예시
컨테이너	데이터를 저장하는 자료구조	`vector`, `list`, `map`, `set`
반복자	컨테이너 원소를 가리키는 도구	`begin()`, `end()`
알고리즘	정렬, 탐색 같은 기능	`sort`, `find`, `count`

subject	title	check ☐ ☐ ☐ ☐ ☐
수요일스터디	수요일스터디 - C++ 조사 주제 4개 개념정리	date

vector

`vector`는 크기가 자동으로 늘어나는 배열입니다. 데이터를 순서대로 저장하고, 인덱스를 이용해 빠르게 접근할 수 있습니다.

```
#include <vector>
using namespace std;

vector<int> scores;
scores.push_back(80);
scores.push_back(90);
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

일반 배열은 크기를 미리 정해야 하지만, `vector`는 데이터가 늘어나면 크기를 자동으로 조절합니다. 순서가 있고, 번호로 바로 꺼내야 하는 데이터에 적합합니다.

주요 사용처:

- 학생 점수 목록처럼 데이터 개수가 계속 늘어날 수 있는 경우
- 책 목록, 상품 목록, 게시글 목록처럼 순서대로 저장하고 전체를 훑어보는 경우
- 인덱스 번호로 `scores[0]`, `scores[1]`처럼 바로 접근해야 하는 경우

list

`list`는 연결 리스트입니다. 각 데이터가 앞뒤 노드와 연결되어 있어서 중간 삽입과 삭제에 유리합니다.

```
#include <list>
using namespace std;

list<int> numbers;
numbers.push_back(10);
numbers.push_front(5);
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

중간 삽입과 삭제가 자주 일어나는 경우에 사용할 수 있습니다. 다만 `list[0]`처럼 인덱스로 바로 접근하는 것은 `vector`보다 불편합니다.

주요 사용처:

- 대기 명단 중간에 사람을 끼워 넣거나 빼야 하는 경우
- 음악 재생 목록처럼 앞뒤 항목을 연결해서 이동하는 구조
- 데이터 위치가 자주 바뀌고, 중간 삭제가 많은 경우

subject	title	check ☐ ☐ ☐ ☐ ☐
수요일스터디	수요일스터디 - C++ 조사 주제 4개 개념정리	date

stack

`stack`은 LIFO 구조입니다.

LIFO = Last In, First Out

나중에 들어온 값이 먼저 나감

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

즉, 마지막에 넣은 데이터를 가장 먼저 꺼내야 할 때 사용합니다.

```
#include <stack>
using namespace std;

stack<int> s;
s.push(10);
s.push(20);
s.pop();
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

주요 사용처:

- 문서 편집기의 실행 취소 기능
- 웹 브라우저의 뒤로가기 흐름
- 괄호 검사처럼 가장 최근에 열린 괄호를 먼저 확인해야 하는 경우
- 함수 호출처럼 마지막에 들어온 작업이 먼저 끝나는 구조

subject	title	check ☐ ☐ ☐ ☐ ☐
수요일스터디	수요일스터디 - C++ 조사 주제 4개 개념정리	date

queue

`queue`는 FIFO 구조입니다.

FIFO = First In, First Out
먼저 들어온 값이 먼저 나감

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

즉, 먼저 들어온 데이터를 먼저 처리해야 할 때 사용합니다.

```
#include <queue>
using namespace std;

queue<int> q;
q.push(10);
q.push(20);
q.pop();
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

주요 사용처:

- 은행, 병원, 상담센터처럼 접수 순서대로 처리하는 경우
- 프린터 출력 대기열처럼 먼저 요청한 작업을 먼저 처리하는 경우
- 서버 요청을 순서대로 처리하는 작업 큐
- 너비 우선 탐색처럼 가까운 순서대로 차례차례 확인하는 알고리즘

map

`map`은 key와 value를 한 쌍으로 저장하는 연관 컨테이너입니다. key를 기준으로 value를 찾습니다.

```
#include <map>
#include <string>
using namespace std;

map<string, int> score;
score["Kim"] = 90;
score["Lee"] = 85;
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

값을 번호가 아니라 이름이나 아이디 같은 key로 찾고 싶을 때 사용합니다.

```
"Kim" -> 90점
"Lee" -> 85점
"Park" -> 100점
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

여기서 `"Kim"`은 key이고, `90`은 value입니다.

subject	title	check ☐ ☐ ☐ ☐ ☐
수요일스터디	수요일스터디 - C++ 조사 주제 4개 개념정리	date

주요 사용처:

- 학생 이름으로 점수 찾기
- 회원 ID로 회원 정보 찾기
- 상품명으로 가격 찾기
- 단어와 뜻을 연결하는 사전 만들기
- 파일 확장자와 실행 프로그램을 연결하는 설정표 만들기

set

`set`은 중복을 허용하지 않는 컨테이너입니다. 같은 값을 여러 번 넣어도 하나만 저장됩니다.

```
#include <set>
using namespace std;

set<int> studentIds;
studentIds.insert(1001);
studentIds.insert(1001);
studentIds.insert(1002);
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

`1001`을 두 번 넣어도 한 번만 저장됩니다.

주요 사용처:

- 중복 없는 출석 번호 목록 만들기
- 이미 방문한 위치나 페이지를 기록하기
- 중복 단어를 제거하고 단어 목록 만들기
- 같은 회원 ID가 두 번 들어가지 않게 관리하기

sort

`sort()`는 지정한 범위의 데이터를 정렬하는 알고리즘입니다.

```
#include <algorithm>
#include <vector>
using namespace std;

vector<int> score = {50, 90, 70, 100};
sort(score.begin(), score.end());
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

결과

결과

50 70 90 100

subject	title	check ☐ ☐ ☐ ☐ ☐
수요일스터디	수요일스터디 - C++ 조사 주제 4개 개념정리	date

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

기본적으로 오름차순으로 정렬하며, 비교 함수를 사용하면 원하는 기준으로 정렬할 수 있습니다.

주요 사용처:

- 시험 점수를 오름차순 또는 내림차순으로 정렬하기
- 책 목록을 제목순으로 정렬하기
- 회원 목록을 가입일순으로 정렬하기
- 상품 목록을 가격순으로 정렬하기
- 검색 전에 데이터를 정렬해 이진 탐색을 준비하기

구조체 정렬

숫자뿐 아니라 구조체도 원하는 기준으로 정렬할 수 있습니다.

```
#include <algorithm>
#include <string>
#include <vector>
using namespace std;

struct Student {
    string name;
    int score;
};

bool compare(Student a, Student b) {
    return a.score > b.score;
}

int main() {
    vector<Student> students = {
        {"Kim", 90},
        {"Lee", 80},
        {"Park", 100}
    };

    sort(students.begin(), students.end(), compare);
}
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

`compare()` 함수는 점수가 높은 학생이 앞으로 오도록 정렬 기준을 정합니다.

subject	title	check ☐ ☐ ☐ ☐ ☐
수요일스터디	수요일스터디 - C++ 조사 주제 4개 개념정리	date

iterator

iterator는 컨테이너 안의 원소를 가리키는 도구입니다. 포인터와 비슷하게 생각하면 됩니다.

```
vector<int> v = {10, 20, 30};

for (vector<int>::iterator it = v.begin(); it != v.end(); it++) {
    cout << *it << endl;
}
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

`begin()`은 첫 번째 원소를 가리키고, `end()`는 마지막 다음 위치를 가리킵니다.

주요 사용처:

- `vector`, `list`, `set`, `map` 같은 STL 컨테이너를 공통된 방식으로 순회할 때
- `sort(v.begin(), v.end())`처럼 알고리즘 함수에 범위를 알려줄 때
- 컨테이너 내부 구조를 자세히 몰라도 처음부터 끝까지 값을 확인해야 할 때

STL을 쓰는 이유

자료구조와 알고리즘을 직접 만들면 시간이 오래 걸리고 오류가 생기기 쉽습니다. STL은 이미 검증된 도구를 제공하므로 개발 속도와 안정성을 높여줍니다.

C로 도서관 프로그램을 만들면 배열, 문자열 비교, 정렬, 탐색을 직접 구현해야 합니다.

C++로 만들면 다음처럼 STL을 사용할 수 있습니다.

```
vector<Book> books;
sort(books.begin(), books.end(), compare);
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

대표 컨테이너 선택법

상황	추천 STL
순서대로 여러 데이터를 저장하고 싶다	`vector`
중간 삽입, 삭제가 많다	`list`
최근 작업부터 꺼내고 싶다	`stack`
들어온 순서대로 처리하고 싶다	`queue`
이름이나 ID로 값을 찾고 싶다	`map`
중복 없는 값만 저장하고 싶다	`set`
데이터를 정렬하고 싶다	`sort()`

subject	title	check ☐ ☐ ☐ ☐ ☐
수요일스터디	수요일스터디 - C++ 조사 주제 4개 개념정리	date

실제 프로그램에서의 사용 예

프로그램 상황	사용하면 좋은 STL	이유
학생 여러 명의 점수를 저장한다	<code>`vector`</code>	학생 수가 늘어나도 자동으로 크기가 조절됨
접수한 순서대로 상담을 처리한다	<code>`queue`</code>	먼저 들어온 요청을 먼저 처리할 수 있음
실행 취소 기능을 만든다	<code>`stack`</code>	가장 최근 작업부터 되돌릴 수 있음
이름으로 점수를 찾는다	<code>`map`</code>	이름을 key로 해서 점수를 바로 찾을 수 있음
중복 없는 회원 ID를 관리한다	<code>`set`</code>	같은 ID가 여러 번 들어가는 것을 막을 수 있음
책 목록을 제목순으로 정렬한다	<code>`sort()`</code>	원하는 기준으로 목록을 보기 좋게 정렬할 수 있음

최종 암기 문장

STL은 C++에서 표준으로 제공하는 자료구조와 알고리즘 라이브러리입니다. ``vector``는 자동으로 커지는 배열, ``stack``은 LIFO, ``queue``는 FIFO, ``map``은 key-value 저장, ``set``은 중복 없는 저장, ``sort``는 정렬에 사용합니다.

subject	title	check ☐ ☐ ☐ ☐ ☐
수요일스터디	수요일스터디 - C++ 조사 주제 4개 개념정리	date

전체 최종 정리

4개 주제 한 번에 암기

주제	핵심 문장
C와 C++ 차이	C는 절차지향 중심, C++은 C에 객체지향과 STL을 더한 언어
Call 방식	Value는 복사본, Address는 주소, Reference는 별명
C++과 Java 차이	C++은 성능과 직접 제어, Java는 JVM 기반 안정성과 유지보수
STL	C++에서 표준으로 제공하는 자료구조와 알고리즘 라이브러리

발표용 마무리 문장

C와 C++의 차이는 단순한 문법 차이가 아니라 프로그램을 설계하는 방식의 차이입니다. C++에서는 함수 전달 방식, 클래스, 객체지향, STL을 이해해야 더 효율적인 코드를 작성할 수 있습니다. 또한 C++과 Java는 둘 다 객체지향을 지원하지만, C++은 성능과 직접 제어에 강하고 Java는 JVM 기반 실행과 유지보수에 강하다는 차이가 있습니다.

확인 질문

1. C와 C++의 가장 큰 차이는 무엇인가? 2. 'struct'와 'class'는 어떤 점이 다른가? 3. Call by Value에서 원본 값이 바뀌지 않는 이유는 무엇인가? 4. Call by Address와 Call by Reference의 공통점은 무엇인가? 5. 'const reference'는 왜 사용하는가? 6. C++에서 'virtual'이 중요한 이유는 무엇인가? 7. Java가 여러 운영체제에서 실행되기 쉬운 이유는 무엇인가? 8. C++과 Java의 메모리 관리 방식은 어떻게 다른가? 9. STL이 필요한 이유는 무엇인가? 10. 'vector', 'stack', 'queue', 'map', 'set'은 각각 언제 사용하는가?